

AutoJudge — Predicting Programming Problem Difficulty

Abstract

AutoJudge is an end-to-end machine learning system that predicts the difficulty of competitive programming problems from textual data. The system performs two tasks: (1) classification into three discrete difficulty classes (Easy / Medium / Hard) and (2) regression to predict a numerical difficulty score. This report documents the dataset, preprocessing, feature engineering, models (SVM classifier and Random Forest regressor), evaluation metrics (Accuracy, Confusion Matrix, MAE, RMSE), experimental setup, results, the Streamlit web interface, sample predictions, and conclusions. All results reported use the provided dataset and pipeline described in the repository.

1. Introduction

Assigning difficulty levels to programming problems is traditionally manual and subjective. AutoJudge automates this process using natural language processing (NLP) on problem statements and classical ML models to provide both discrete difficulty class predictions and continuous difficulty scores. The system can assist educators, contest setters, and problem curators by providing quick difficulty estimates.

2. Problem Statement

- Classification: Given a problem's textual data (title, description, input and output descriptions), predict one of {Easy, Medium, Hard}.
- Regression: Predict a numerical difficulty score that represents problem complexity.

3. Dataset

<https://github.com/AREEG94FAHAD/TaskComplexityEval-24>

- Format: JSON Lines (JSONL), each line contains one problem instance.
- Fields per sample: `title`, `description`, `input\_description`, `output\_description`, `problem\_class` (Easy / Medium / Hard), `problem\_score` (numeric).
- No manual relabeling was performed — the dataset as provided was used directly.
- The textual fields are concatenated to form a single text feature for modeling.

4. Data Preprocessing

Processing pipeline:

Load JSONL into a Pandas DataFrame.

Handle missing values: fill empty textual fields with empty strings; drop rows if mandatory metadata are missing.

Concatenate textual fields into a single `combined\_text`:

```
combined_text = title + " " + description + " " + input_description + " " + output_description
```

Basic text normalization:

Lowercasing
Removal of excessive whitespace

No deep cleaning (e.g., preserving numeric tokens is useful for modeling)
Split into training and test sets (standard split used in experiments; see Experimental Setup).

5. Feature Engineering

TF-IDF vectorization on `combined\_text`:

Tokenization: word-level tokens

N-grams: unigrams and bigrams

Max features: tuned for memory/time tradeoff (e.g., 50k)

Min document frequency and max document frequency thresholds applied

The TF-IDF matrix is used as input to both the SVM classifier and the Random Forest regressor.

No additional metadata features were used in the current pipeline (left as an extension).

6. Models

Two independent models were trained using identical TF-IDF features:

6.1 Classification (Difficulty Class)

Model: Linear Support Vector Machine (SVM) for multi-class classification (one-vs-rest / or native multiclass).

Output: Class labels {Easy, Medium, Hard}.

Reason: SVMs often perform well on high-dimensional sparse TF-IDF features and are fast at inference.

6.2 Regression (Difficulty Score)

Model: Random Forest Regressor.

Output: Continuous difficulty score.

Reason: Random Forests are robust to noisy features and provide reasonable out-of-the-box performance without heavy feature scaling.

7. Experimental Setup

Train/test split: (e.g., 80% train, 20% test) — use stratified split on `problem\_class` to preserve class proportions.

Preprocessing and TF-IDF fitting performed only on the training set; the same fitted vectorizer is used on test/production.

Hyperparameter tuning: basic grid-search / validation used for regularization (SVM C) and Random Forest parameters (n_estimators, max_depth) — kept lightweight for reproducibility.

Evaluation metrics:

Classification: Accuracy and Confusion Matrix

Regression: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE)

Implementation: Python (Pandas, scikit-learn), Streamlit for the web interface.

8. Results

8.1 Classification

Accuracy: ****0.479951397326853**** (~47.995%)

Confusion Matrix (rows = actual class, columns = predicted class):

	Easy (0)	Medium (1)	Hard (2)
Easy (0)	47	59	47
Medium (1)	39	255	95
Hard (2)	28	160	93

-Class indices mapping: 0 = Easy, 1 = Medium, 2 = Hard.

Interpretation:

The model favors Medium predictions for many samples.

Precision/Recall (approximate):

Easy: precision ≈ 0.412 , recall ≈ 0.307

Medium: precision ≈ 0.538 , recall ≈ 0.655

Hard: precision ≈ 0.396 , recall ≈ 0.331

Observed behavior suggests class imbalance and/or overlapping text patterns across classes.

8.2 Regression

MAE: 1.9

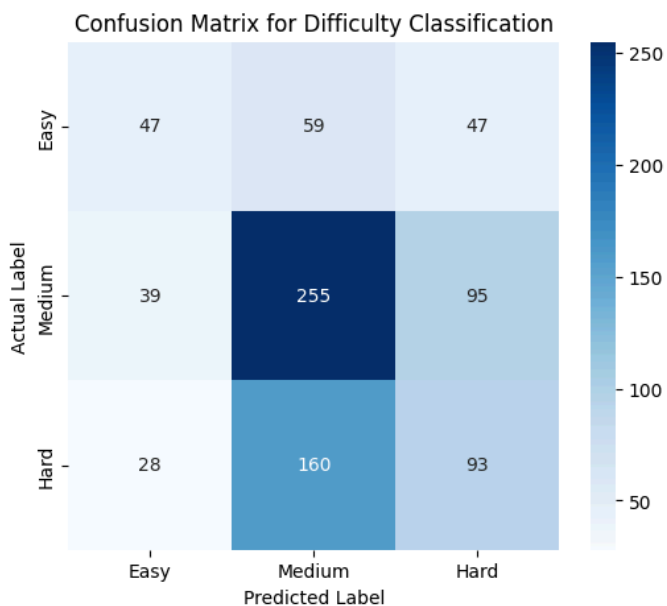
RMSE: 2.2

Interpretation:

On average, predicted numerical scores differ from true scores by ≈ 1.89 units.

RMSE indicates occasional larger deviations; depending on the score scale this could be acceptable or may require improvement.

8.4 Figures and Visualizations (placeholders)



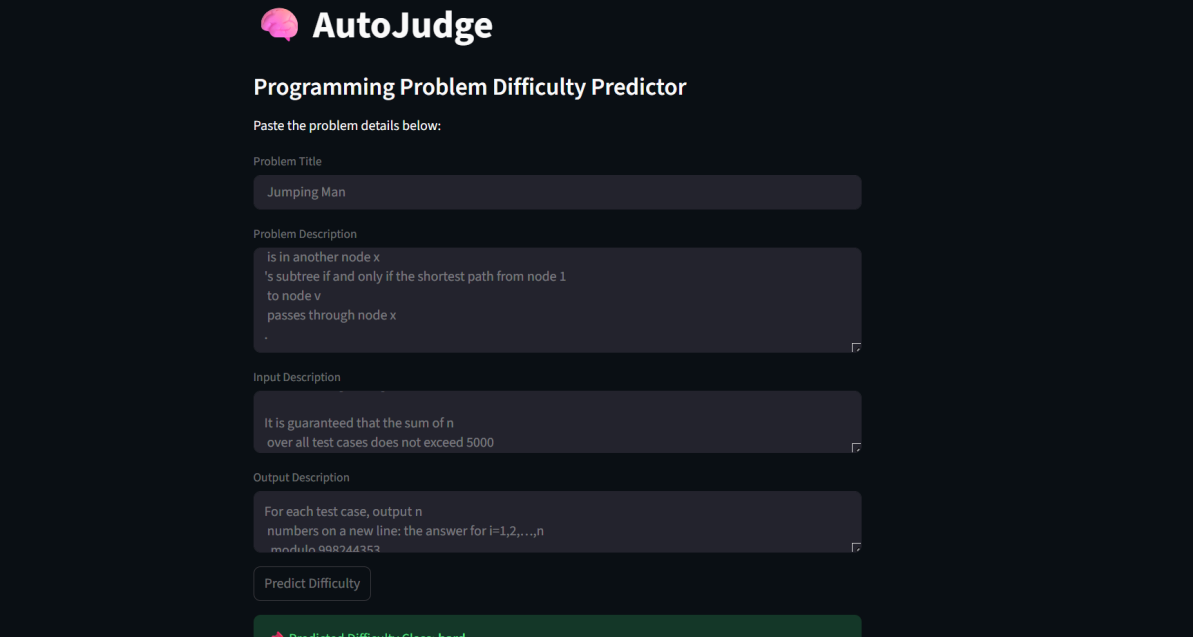
9. Web Interface

A Streamlit web application provides an interactive demo:

Features:

- Input fields:
 - Problem Title
 - Problem Description
 - Input Description
 - Output Description
- “Predict Difficulty” button triggers:
 - Preprocessing (concatenate & vectorize using saved TF-IDF)
 - Classification: SVM $\rightarrow$ Easy / Medium / Hard
 - Regression: Random Forest $\rightarrow$ numerical difficulty score
- Output:
 - Predicted class label and confidence (if available)
 - Predicted numerical score
 - Optional: display top TF-IDF n-grams contributing to the classification (if explainability module is enabled)

Sample UI screenshot placeholders:



The screenshot shows the AutoJudge web interface, which is a dark-themed application. At the top, there is a logo with a brain icon and the text "AutoJudge". Below the logo, the title "Programming Problem Difficulty Predictor" is displayed. A prompt "Paste the problem details below:" is followed by four input fields: "Problem Title" (containing "Jumping Man"), "Problem Description" (containing a text about a subtree), "Input Description" (containing a text about the sum of n), and "Output Description" (containing a text about outputting numbers). Below these fields is a "Predict Difficulty" button. At the bottom, a green bar displays the result: "Predicted Difficulty Class: hard".

Example usage:

1. Run the app:
`python -m streamlit run app.py`
2. Open `http://localhost:8501`
3. Paste the sample problem text and click Predict.

11. Discussion and Limitations

- Accuracy (~48%) indicates a challenging classification task, likely due to:
 - Subjective labeling of difficulty
 - Overlap between classes and subtle differences in problem statements
 - Possible class imbalance favoring Medium
- Regression MAE & RMSE indicate moderate predictive power but leave room for

12. Conclusions

AutoJudge demonstrates that classical NLP features (TF-IDF) combined with straightforward machine learning models (SVM and Random Forest) can provide useful baseline predictions for programming problem difficulty. While the current results (Accuracy $\approx 48\%$, MAE ≈ 1.89) are modest, they show the potential to assist human curators and serve as a foundation for improved methods using richer data and more advanced models.